

CS & IT ENGINEERING

'C' Programming

Functions

Lecture No.- 03



By- Satya sir

Recap of Previous Lecture



- Static Scoping vs Dynamic Scoping

- Recursion

- caller function == callee function

- Types of Recursion

- Direct Recursion

- Indirect Recursion

Head Recursion ✓

Tail Recursion ✓

Tree Recursion

Nested Recursion

Topics to be Covered



- Tree Recursion
- Nested Recursion
- Indirect Recursion
- Storage classes



Topic : Recursion Examples



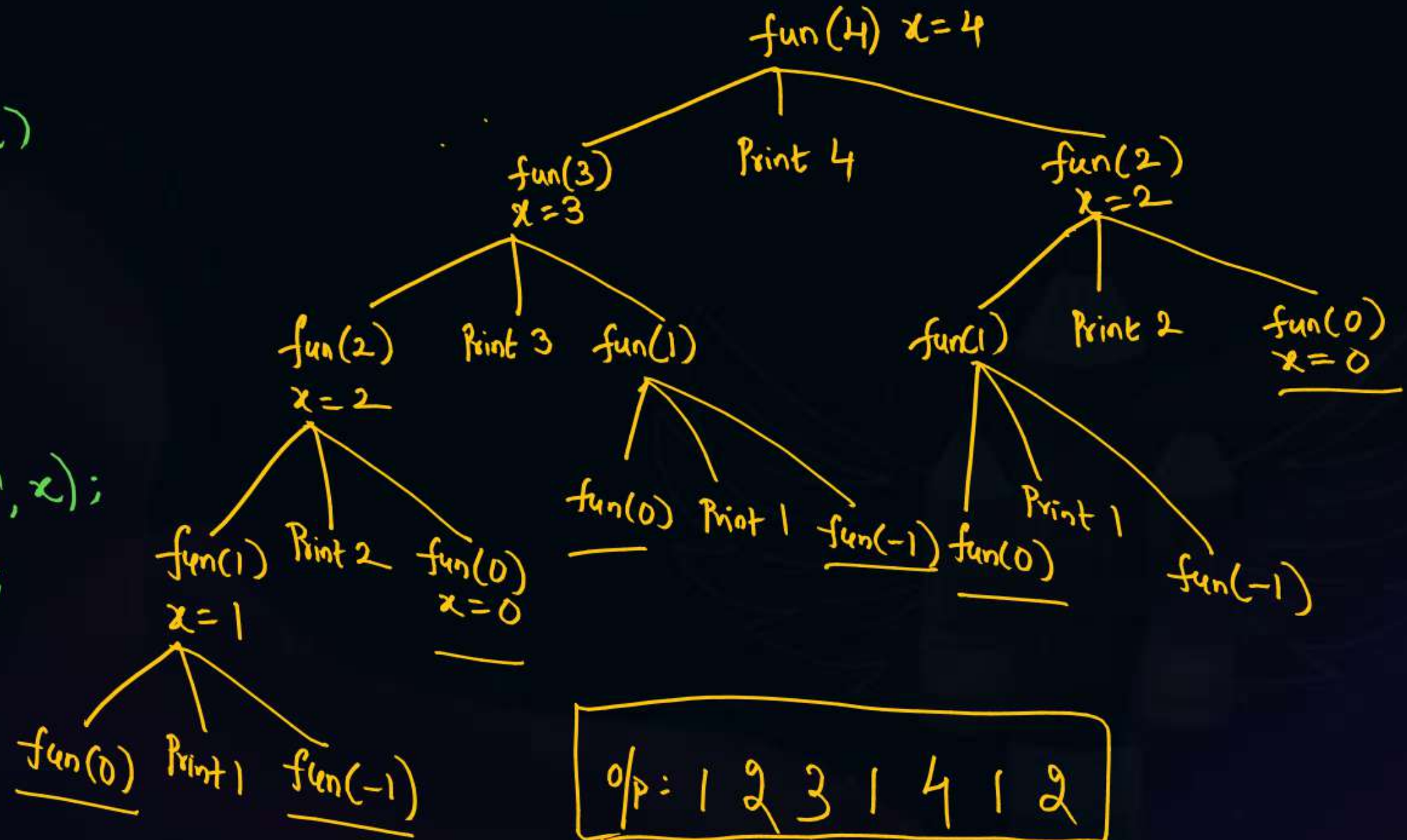
Tree Recursion

- when Recursive call is made more than one time, Then Such recursion is said to be Tree Recursion.

Ex:

```
Void fun(int x)
{
    if (x <= 0)
        return;
    fun(x-1);
    Printf("-/-d", x);
    fun(x-2);
}

Void main()
{
    fun(4);
}
```





Topic : Recursion Examples



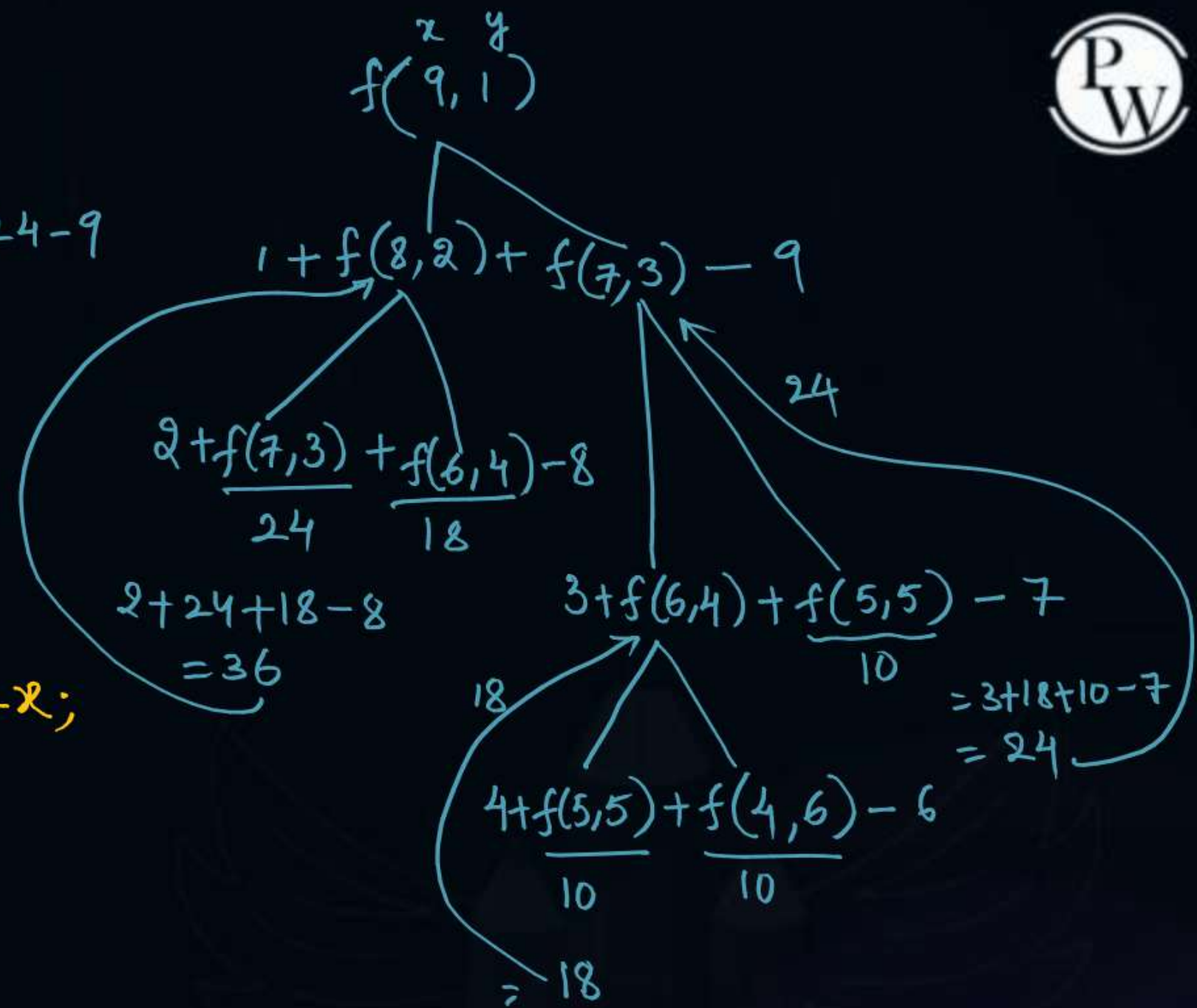
Ex: 2

```
int f(int x, int y)
{
    if(x <= y)
        return x+y;
    return y + f(x-1, y+1) + f(x-2, y+2) - x;
}
```

→ Void main()
{ int a=9, b=1;
 printf("%d", f(a, b));
}

o/p = 52

$$1 + 36 + 24 - 9 = 52$$





Topic : Recursion Examples



Nested Recursion

If, in one Recursive call another recursive call, that in another and so on is made,

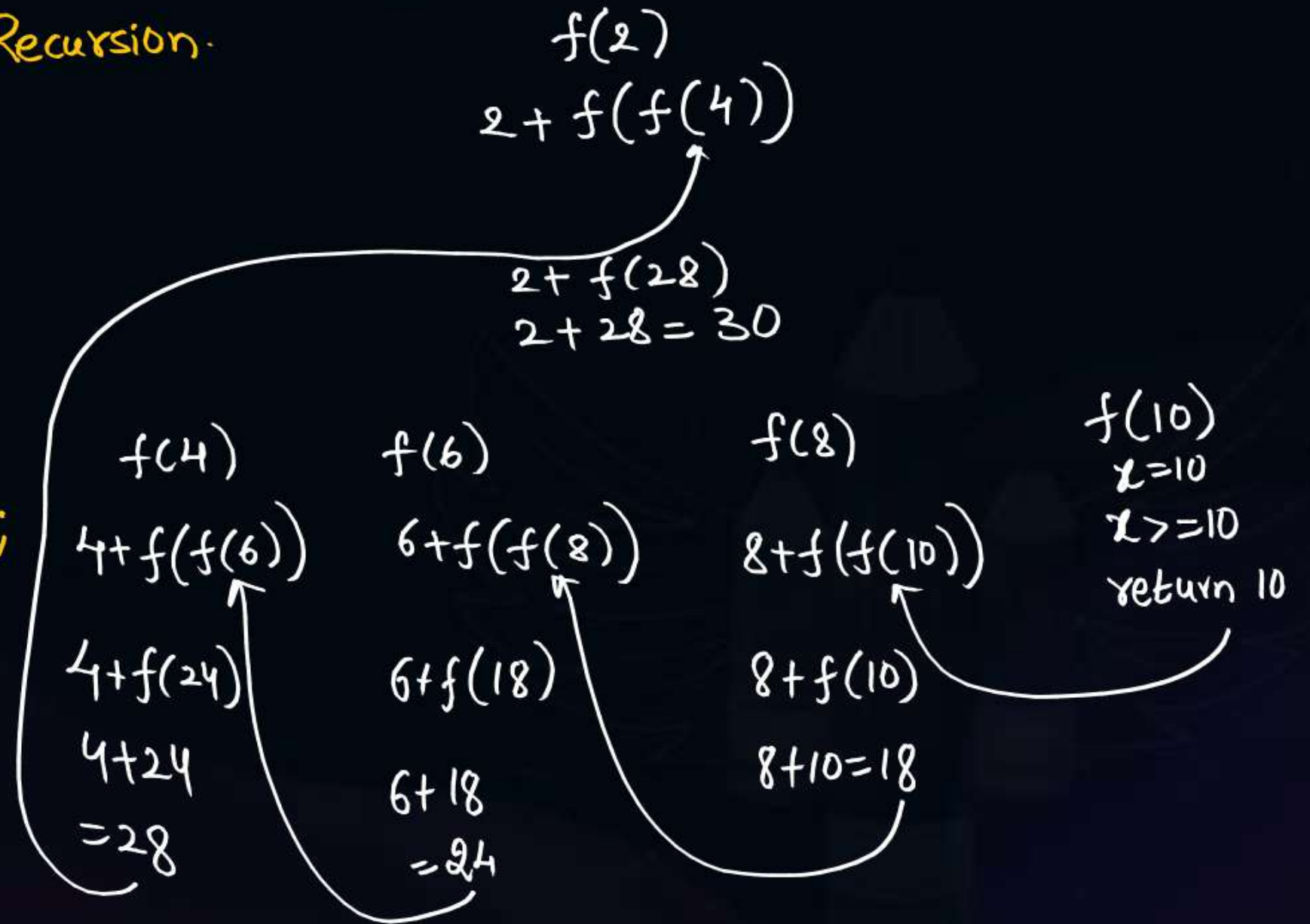
Such recursion is said to be Nested Recursion.

Ex:1

```
int f(int x)
{
    if (x >= 10)
        return x;
    return x + f(f(x+2));
}

Void main()
{
    Printf("%d", f(2));
}
```

o/p: 30





Topic : Recursion Examples



Ex: 2

```
int fun(int x)
{
    if (x <= 1)
        return x;
```

```
    return x + fun(fun(fun(x >> 2)));
}
```

```
Void main ( )
```

```
{
    int x = 1000;
```

```
    printf("%d", fun(x));
}
```

$f(72)$

$72 + f(f(f(18)))$

$72 + f(f(25))$

$72 + f(10)$

$72 + 12$
 $= 84$

$f(10)$

$10 + f(f(f(2)))$

$10 + 2 = 12$

$f(84)$

$84 + f(f(f(21)))$

$84 + f(f(29))$

H/W

$fun(1000)$

$1000 + f(f(f(250)))$

$f(2) = 2$

$f(4)$

$4 + f(f(f(1)))$

$4 + 1$
 $= 5$

$f(5) = 5 + 1$
 $= 6$

$f(6) = 6 + 1 = 7$
 $f(7) = 7 + 1 = 8$

$f(18)$

$18 + f(f(f(4)))$

$18 + f(f(5))$

$18 + f(6)$

$18 + 7 = 25$

$f(25)$

$x = 25$

$25 + f(f(f(6)))$

$25 + f(f(7))$

$25 + f(8)$

$8 + f(f(f(2)))$

$8 + f(f(2))$

$8 + 2 = 10$

$f(250)$

$250 + f(f(f(62)))$

$250 + f(f(72))$

$250 + f(87)$

$f(12)$

$12 + f(f(f(3)))$

$12 + 3$

$= 15$

$f(21)$

$21 + f(f(f(5)))$

$21 + f(f(6))$

$21 + f(7)$

$21 + 8$

$= 29$

$f(62)$

$62 + f(f(f(15)))$

$62 + f(f(18))$

$62 + f(25)$

$62 + 10 = 72$

$f(15)$

$15 + f(f(f(3)))$

$15 + f(f(3))$

$15 + f(3)$

$15 + 3$

$= 18$

$f(3)$

$3 + f(f(f(0)))$

$3 + f(f(0))$

$3 + f(0)$

$3 + 0$

$= 3$



Topic : Storage Classes



Ex: 2

```
int f(int x)
{
    if (x >= 100)
        return x + 2;
    return x + f(f(x * 3));
}

Void main( )
{
    Printf("%d", f(10));
}
```

o/p : 408

$$\begin{aligned} &f(10) \\ &10 + f(f(30)) \\ &10 + f(396) \\ &10 + 398 \\ &= \underline{\underline{408}} \end{aligned}$$

$$\begin{aligned} &f(30) \\ &30 + f(f(90)) \\ &30 + f(364) \\ &30 + 366 \\ &= 396 \end{aligned}$$

$$\begin{aligned} &f(90) \\ &90 + f(f(270)) \\ &90 + f(272) \\ &90 + 274 \\ &= 364 \end{aligned}$$



Topic : Storage Classes



Indirect Recursion

A function calls itself, through an intermediate function.

Ex:

```
Void A(int i)
```

```
{
```

```
    if (i/.2)
```

→ odd i value == True

```
    return
```

```
        B(i+3);
```

```
}
```

```
Void B(int j)
```

```
{
```

```
    if (j/.3 == 0)
```

```
        return
```

```
        printf("%d", j); ✓
```

```
}
```

```
    A(j+1); ✓
```

→ Void main()

```
{
```

```
    A(10);
```

```
}
```

o/p: 13 17

main()

↓ i

A(10)

↓ j

B(13) ✓

Print 13

↑ i

A(14)

↓ j

↳ B(17)

Print 17

↑ i

A(18)

↓ j

↳ B(21)

21/.3 == 0 True



Topic : Storage Classes



Storage classes

{ int i; // Where memory is allocated for i? [RAM, ROM, HD, CPU Registers]

printf("%d", i); // What will be default value?
}

// What will be scope of Variable?

// What will be lifetime of Variable?

Storage class defines

- 1) Default value
- 2) Memory location
- 3) Scope and
- 4) Lifetime of a Variable/data.

Syntax: Storage class datatype Variable;
 keyword

There are 4 storage classes in 'C':

		keyword	Example
* [default]	1) Automatic storage class :	auto	auto int x;
	2) Register	register	register int x;
	3) Static	static	static int x;
	4) External	extern	extern int x;



Topic : Storage Classes



Storage class	Default value	Memory Location	Scope	Lifetime
Automatic	Garbage (or) random value	RAM (main memory)	Block scope	Within Block
Register	Garbage (or) random value	CPU Registers (or) RAM	Block scope	Within Block
Static	Zero	RAM	Block scope	Throughout Program Execution
External	Zero	RAM	File scope	Throughout Program Directory.



2 mins Summary



- Tree, Nested, indirect Recursion
- Storage classes - auto, register, static

auto/register Vs static

Void fun()

{ auto int x=5; (or) register int x=5;

x=x+1;

Printf(" %.d", x);



Void main()

{ int i;

for(i=1; i<5; i++)

fun();

}

o/p: 6 6 6 6

i=1 fun();

i=2 fun();

i=3 fun();

i=4 fun();

Void fun()

{

static int x=5;

x=x+1;

Printf(" %.d", x);

}

Void main()

{

int i;

for(i=1; i<5; i++)

fun();

}

Static variables are
Declared and initialized only
once, first time.

~~x 5 6 7 8 9~~

i=1 fun();

i=2 fun();

i=3 fun();

i=4 fun();

o/p: 6 7 8 9



NOTE: Structures and Unions — Single shot.

THANK - YOU